

LAPORAN PRAKTIKUM STRUKTUR DATA

**Single Linked List**

DISUSUN OLEH:

Rayya Syaquinah Putri Hasibuan

2511532007

DOSEN PENGAMPU:

Dr. Wahyudi, S.T. M.T

ASISTEN PRATIKUM:

M. Galid



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

## KATA PENGANTAR

Puji syukur penulis panjatkan kehadiran Tuhan Yang Maha Esa atas segala rahmat dan karunia-Nya sehingga laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban pelaksanaan praktikum struktur data dengan judul Single Linked List. Laporan ini bertujuan untuk mendokumentasikan prosedur, hasil, serta pembahasan dari kegiatan praktikum sekaligus sebagai sarana pembelajaran bagi penulis untuk lebih memahami materi terkait.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu. Saran dan kritik yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu dan semua pihak yang telah membantu sehingga laporan ini dapat terselesaikan.

Padang, 4 Mei 2026

Rayya Syaquinah Putri Hasibuan

## DAFTAR ISI

|                                     |     |
|-------------------------------------|-----|
| KATA PENGANTAR.....                 | ii  |
| DAFTAR ISI .....                    | iii |
| BAB I PENDAHULUAN .....             | 1   |
| 1.1    Latar Belakang .....         | 1   |
| 1.2    Tujuan Praktikum.....        | 1   |
| 1.3    Manfaat Praktikum.....       | 1   |
| BAB II PEMBAHASAN .....             | 2   |
| 2.1    Landasan Teori .....         | 2   |
| 2.2    Prosedur Praktikum.....      | 3   |
| a.    NodeSLL_2511532007 .....      | 3   |
| b.    TambahSLL_2511532007.....     | 4   |
| c.    PencarianSLL_2511532007 ..... | 7   |
| d.    HapusSLL_2511532007.....      | 8   |
| BAB III KESIMPULAN DAN SARAN .....  | 11  |
| 3.1    Kesimpulan .....             | 11  |
| 3.2    Saran.....                   | 11  |
| DAFTAR PUSTAKA .....                | 12  |
| TUGAS PRAKTIKUM PEKAN 5 .....       | 13  |

## **BAB I**

### **PENDAHULUAN**

#### **1.1 Latar Belakang**

Struktur data merupakan konsep dasar dalam pemrograman yang digunakan untuk mengelola dan menyimpan data secara efisien. Salah satu struktur data yang penting untuk dipahami adalah Single Linked List, karena memiliki keunggulan dalam pengelolaan data yang bersifat dinamis. Berbeda dengan array yang memiliki ukuran tetap, linked list memungkinkan penambahan dan penghapusan data tanpa harus memindahkan elemen lainnya. Dalam praktikum ini, dilakukan implementasi operasi dasar pada Single Linked List seperti penambahan, pencarian, dan penghapusan data. Pemahaman terhadap konsep ini menjadi penting karena sering digunakan dalam pengembangan program yang membutuhkan fleksibilitas dalam pengolahan data.

#### **1.2 Tujuan Praktikum**

Praktikum ini bertujuan untuk memahami konsep dasar Single Linked List serta cara kerja penyimpanan data secara berurutan. Selain itu, praktikum ini juga bertujuan untuk melatih kemampuan dalam mengimplementasikan operasi-operasi dasar seperti menambah data, mencari data, dan menghapus data dalam suatu struktur linked list. Dengan demikian, mahasiswa diharapkan mampu memahami alur kerja program serta mampu mengaplikasikan konsep tersebut dalam pembuatan program yang lebih kompleks.

#### **1.3 Manfaat Praktikum**

Manfaat dari praktikum ini adalah mahasiswa dapat memahami cara kerja struktur data Single Linked List secara lebih mendalam, baik dari segi konsep maupun implementasi. Selain itu, praktikum ini juga membantu meningkatkan kemampuan logika pemrograman dalam mengelola data secara dinamis. Dengan memahami linked list,

mahasiswa akan lebih siap dalam mempelajari struktur data lainnya serta mampu mengembangkan solusi pemrograman yang lebih efisien dan fleksibel dalam berbagai kasus.

## **BAB II**

### **PEMBAHASAN**

#### **2.1 Landasan Teori**

Struktur data merupakan cara yang digunakan untuk mengorganisasi dan menyimpan data agar dapat diakses serta diolah secara efisien dalam suatu program. Salah satu jenis struktur data linier yang sering digunakan adalah linked list. Linked list adalah kumpulan elemen yang disebut node, di mana setiap node menyimpan data dan terhubung dengan elemen lainnya dalam suatu urutan tertentu. Berbeda dengan array, linked list tidak menyimpan data secara bersebelahan dalam memori, melainkan tersebar dan saling terhubung melalui referensi antar elemen sehingga memberikan fleksibilitas dalam pengelolaan data.

Setiap node dalam linked list terdiri dari dua bagian utama, yaitu bagian yang menyimpan data dan bagian yang menunjukkan hubungan ke elemen berikutnya. Node pertama disebut sebagai head yang menjadi titik awal dari linked list, sedangkan node terakhir akan berakhir dengan nilai kosong yang menandakan tidak adanya elemen setelahnya. Dengan adanya konsep ini, data dapat disusun secara berantai dari awal hingga akhir sesuai dengan urutan yang diinginkan.

Linked list memiliki beberapa keunggulan dibandingkan struktur data lain seperti array. Salah satu keunggulannya adalah kemampuannya untuk mengelola data secara dinamis, di mana jumlah data dapat bertambah atau berkurang tanpa perlu menentukan kapasitas awal. Selain itu, proses penambahan dan penghapusan data dapat dilakukan dengan lebih efisien karena tidak memerlukan pergeseran elemen lain. Namun, linked list juga memiliki kelemahan, yaitu proses pencarian data yang harus dilakukan secara berurutan

dari awal hingga akhir, sehingga kurang efisien jika jumlah data sangat besar.

Dalam penerapannya, linked list memiliki beberapa operasi dasar yang penting, yaitu penambahan data (insert), penghapusan data (delete), penelusuran data (traversal), dan pencarian data (search). Operasi-operasi ini menjadi dasar dalam pengolahan data menggunakan linked list. Selain itu, linked list juga dapat dikembangkan menjadi beberapa jenis lain, seperti Single Linked List, Double Linked List, dan Circular Linked List, yang masing-masing memiliki karakteristik dan kegunaan tersendiri.

Pemahaman terhadap linked list sangat penting dalam dunia pemrograman karena struktur ini menjadi dasar bagi berbagai implementasi lain, seperti stack dan queue. Dengan memahami konsep dan cara kerja linked list, seseorang dapat mengembangkan program yang lebih fleksibel dan efisien dalam mengelola data, terutama ketika berhadapan dengan data yang jumlahnya tidak tetap.

## 2.2 Prosedur Praktikum

### a. NodeSLL\_2511532007

```

1 package pekan5_2511532007;
2
3 public class NodeSLL_2511532007 {
4     // node bagian data
5     int data_2007;
6     // pointer ke data berikutnya
7     NodeSLL_2511532007 next_2007;
8     // konstruktor menginisialisasi node dengan data
9     public NodeSLL_2511532007(int data_2007)
10    {
11        this.data_2007= data_2007;
12        this.next_2007= null;
13    }
14 }
15

```

Langkah-langkah susunan kode program beserta fungsinya:

#### 1. Deklarasi awal program (package dan class)

Program diawali dengan deklarasi package pekan5\_2511532007; yang berfungsi untuk mengelompokkan file Java. Selanjutnya dibuat public class NodeSLL\_2511532007 sebagai kelas yang merepresentasikan satu simpul (node) pada struktur data Single Linked List.

## 2. Deklarasi variabel (atribut node)

Pada class dideklarasikan dua variabel, yaitu `data_2007` bertipe integer sebagai penyimpan nilai data, dan `next_2007` bertipe `NodeSLL_2511532007` yang berfungsi sebagai pointer atau penghubung ke node berikutnya.

## 3. Konsep pointer pada node

Variabel `next_2007` digunakan untuk menyimpan referensi ke node lain. Jika bernilai null, maka node tersebut merupakan node terakhir dalam linked list.

## 4. Pembuatan konstruktor

Konstruktor `NodeSLL_2511532007(int data_2007)` digunakan untuk menginisialisasi node saat pertama kali dibuat dengan nilai data yang diberikan melalui parameter.

## 5. Inisialisasi data pada node

Pada konstruktor, nilai `data_2007` diisi menggunakan `this.data_2007 = data_2007;` yang menunjukkan bahwa data node diambil dari parameter input.

## 6. Inisialisasi pointer node

Variabel `next_2007` diatur bernilai null (`this.next_2007 = null;`) yang menandakan bahwa node belum terhubung dengan node lain.

## 7. Peran node dalam linked list

Node yang telah dibuat akan menjadi elemen dasar dalam pembentukan Single Linked List, di mana setiap node akan saling terhubung melalui pointer `next_2007`.

Dapat disimpulkan bahwa class `NodeSLL_2511532007` merupakan komponen dasar dalam struktur data Single Linked List yang berfungsi untuk menyimpan data serta menghubungkan antar node melalui pointer. Struktur ini memungkinkan data tersusun secara dinamis dan saling terhubung secara berurutan.

### **b. TambahSLL\_2511532007**

```

1 package pekan5_2511532007;
2
3 public class TambahSLL_2511532007 {
4     public static NodeSLL_2511532007 insertAtFront(NodeSLL_2511532007 head_2007, int value_2007) {
5         NodeSLL_2511532007 new_node = new NodeSLL_2511532007(value_2007);
6         new_node.next_2007 = head_2007;
7         return new_node;
8     }
9     // Fungsi menambahkan node diakhir list
10    public static NodeSLL_2511532007 insertAtEnd(NodeSLL_2511532007 head_2007, int value_2007) {
11        // head adalah node dengan pointer awal
12        NodeSLL_2511532007 new_node_2007 = new NodeSLL_2511532007(value_2007);
13        // jika list kosong maka node jadi head
14        if (head_2007 == null) {
15            return new_node_2007;
16        }
17        // simpan head ke variabel sementara
18        NodeSLL_2511532007 last = head_2007;
19        // lakukan ke node akhir
20        while (last.next_2007 != null) {
21            last = last.next_2007;
22        }
23        // temp pointer
24        last.next_2007 = new_node_2007;
25        return head_2007;
26    }
27    static NodeSLL_2511532007 getNode(int data_2007) {
28        return new NodeSLL_2511532007(data_2007);
29    }
30    static NodeSLL_2511532007 insertPos(NodeSLL_2511532007 headNode, int position_2007, int value_2007) {
31        NodeSLL_2511532007 head_2007 = headNode;
32        if (position_2007 < 1) {
33            System.out.println("Invalid position");
34        }
35        if (position_2007 == 1) {
36            NodeSLL_2511532007 new_node = new NodeSLL_2511532007(value_2007);
37            new_node.next_2007 = head_2007;
38            return new_node;
39        } else {

```

```

40        while (position_2007-- > 1) {
41            if (position_2007 == 1) {
42                NodeSLL_2511532007 new_node = getNode(value_2007);
43                new_node.next_2007 = headNode.next_2007;
44                headNode.next_2007 = new_node;
45                break;
46            }
47            headNode = headNode.next_2007;
48        }
49        if (position_2007 != 1) {
50            System.out.println("Posisi di luar jangkauan");
51        }
52        return head_2007;
53    }
54    public static void printList(NodeSLL_2511532007 head_2007) {
55        NodeSLL_2511532007 curr_2007 = head_2007;
56        while (curr_2007 != null) {
57            System.out.print(curr_2007.data_2007 + " ");
58            curr_2007 = curr_2007.next_2007;
59        }
60        System.out.println();
61    }
62    if (curr_2007.next_2007 == null) {
63        System.out.println(curr_2007.data_2007);
64    }
65    System.out.println();
66    }

```

```

67    public static void main(String[] args) {
68        // head linked list di awal
69        NodeSLL_2511532007 head_2007 = new NodeSLL_2511532007(1);
70        head_2007.next_2007 = new NodeSLL_2511532007(3);
71        head_2007.next_2007.next_2007 = new NodeSLL_2511532007(5);
72        head_2007.next_2007.next_2007.next_2007 = new NodeSLL_2511532007(7);
73        // data ke-1
74        System.out.println("sebelum horizontal maui");
75        printList(head_2007);
76        // tambahkan node baru di depan
77        System.out.println("tambah 1 simpul di depan: ");
78        int data_2007 = 1;
79        head_2007 = insertAtFront(head_2007, data_2007);
80        // tampilkan node baru di depan
81        printList(head_2007);
82        // tambahkan node baru di belakang
83        System.out.println("tambah 1 simpul di belakang: ");
84        int data2 = 7;
85        head_2007 = insertAtEnd(head_2007, data2);
86        // tampilkan node baru di belakang
87        printList(head_2007);
88        // tambahkan node baru di tengah
89        System.out.println("tambah 1 simpul ke data ke 4: ");
90        int data3 = 4;
91        int pos = 4;
92        head_2007 = insertPos(head_2007, pos, data3);
93        // tampilkan node baru di tengah
94        printList(head_2007);
95    }

```

Langkah-langkah susunan kode program beserta fungsinya:

1. Deklarasi awal program (package dan class)  
Program diawali dengan package pekan5\_2511532007; untuk mengelompokkan file Java. Kemudian dibuat public class TambahSLL\_2511532007 sebagai kelas utama yang berisi implementasi penambahan data pada Single Linked List.
2. Method insertAtFront() (menambah data di depan)  
Method ini digunakan untuk menambahkan data di bagian awal linked list. Program membuat data baru, kemudian menempatkannya di posisi paling depan sehingga data tersebut menjadi awal dari susunan yang baru.
3. Method insertAtEnd() (menambah data di belakang)  
Method ini digunakan untuk menambahkan data di bagian akhir linked list. Jika data masih kosong, maka data baru langsung menjadi



data pertama. Jika sudah ada data, program akan menelusuri hingga bagian terakhir, kemudian menambahkan data baru di bagian tersebut.

4. Method `GetNode()` (membuat data baru)

Method ini digunakan untuk membuat data baru dengan nilai tertentu. Fungsinya untuk mempermudah pembuatan data saat akan disisipkan ke dalam linked list.

5. Method `insertPos()` (menambah data pada posisi tertentu)

Method ini digunakan untuk menambahkan data pada posisi tertentu dalam linked list. Jika posisi yang diberikan tidak valid, program akan menampilkan pesan kesalahan. Jika posisi berada di awal, data akan ditambahkan di depan. Jika di posisi lain, program akan menelusuri hingga posisi yang diinginkan, kemudian menyisipkan data baru pada urutan tersebut.

6. Method `printList()` (menampilkan isi linked list)

Method ini digunakan untuk menampilkan seluruh data dalam linked list. Data ditampilkan secara berurutan dari awal hingga akhir dengan tanda penghubung antar data.

7. Method `main()` (program utama)

Method main digunakan untuk menjalankan program. Pertama, dibuat data awal yaitu 2, 3, 5, dan 6. Kemudian data tersebut ditampilkan. Setelah itu dilakukan beberapa operasi penambahan, yaitu menambah data di depan (1), menambah data di belakang (7), dan menambah data pada posisi tertentu (4 pada posisi ke-4). Setiap perubahan yang terjadi langsung ditampilkan untuk melihat hasilnya.

8. Alur kerja program

Program dimulai dengan menampilkan data awal, kemudian melakukan beberapa proses penambahan data pada berbagai posisi. Setelah setiap proses, hasil susunan data akan diperbarui dan ditampilkan kembali.

Kesimpulannya, program ini menunjukkan proses penambahan data pada Single Linked List di berbagai posisi, yaitu di awal, di akhir, dan

di tengah. Dengan cara ini, data dapat disusun secara fleksibel dan dinamis tanpa harus bergantung pada ukuran tetap seperti pada array.

### c. PencarianSLL\_2511532007

```

1 package pekan5_2511532007;
2
3 public class PencarianSLL_2511532007 {
4     static boolean searchKey (NodeSLL_2511532007 head, int key) {
5         NodeSLL_2511532007 curr = head;
6         while (curr != null) {
7             if (curr.data_2007 == key)
8                 return true;
9             curr = curr.next_2007;
10        }
11        return false;
12    }
13    public static void traversal(NodeSLL_2511532007 head) {
14        // mulai dari head
15        NodeSLL_2511532007 curr = head;
16        // jika data kosong, return null
17        while (curr != null) {
18            System.out.print(" " + curr.data_2007);
19            curr = curr.next_2007;
20            System.out.println();
21        }
22    }
23    public static void main(String[] args) {
24        NodeSLL_2511532007 head = new NodeSLL_2511532007(10);
25        head.next_2007 = new NodeSLL_2511532007(11);
26        head.next_2007.next_2007 = new NodeSLL_2511532007(12);
27        head.next_2007.next_2007.next_2007 = new NodeSLL_2511532007(13);
28        head.next_2007.next_2007.next_2007.next_2007 = new NodeSLL_2511532007(14);
29        System.out.print("Pencarian SLL : ");
30        traversal(head);
31        // data yang akan dicari
32        int key = 9;
33        System.out.print("Carilah data " + key + " : ");
34        if (searchKey(head, key))
35            System.out.println("Ya");
36        else
37            System.out.println("Tidak ada");
38    }
39 }

```

Langkah-langkah susunan kode program beserta fungsinya:

1. Deklarasi awal program (package dan class)  
Program diawali dengan package pekan5\_2511532007; untuk mengelompokkan file Java. Selanjutnya dibuat public class PencarianSLL\_2511532007 sebagai kelas utama yang digunakan untuk mengimplementasikan proses pencarian data pada Single Linked List.
2. Method searchKey() (mencari data pada linked list)  
Method searchKey digunakan untuk mencari suatu nilai dalam linked list. Proses dimulai dari data pertama, kemudian dilanjutkan ke data berikutnya secara berurutan. Jika nilai yang dicari ditemukan, method akan mengembalikan nilai benar (true). Jika hingga akhir data tidak ditemukan, maka akan mengembalikan nilai salah (false).
3. Konsep pencarian data  
Pencarian dilakukan secara berurutan dari awal hingga akhir data. Hal ini dilakukan karena struktur linked list tidak memungkinkan pencarian langsung ke posisi tertentu.
4. Method traversal() (menampilkan isi linked list)  
Method ini digunakan untuk menampilkan seluruh data dalam

linked list. Data ditampilkan satu per satu mulai dari awal hingga akhir secara berurutan.

#### 5. Method main() (program utama)

Method main berfungsi untuk menjalankan program. Pada bagian ini dibuat beberapa data yang disusun menjadi linked list, yaitu 14, 21, 13, 30, dan 10. Kemudian data tersebut ditampilkan menggunakan method traversal. Setelah itu ditentukan nilai yang akan dicari, yaitu 30, lalu dilakukan proses pencarian dan ditampilkan hasilnya.

#### 6. Alur kerja program

Program akan menampilkan seluruh data terlebih dahulu, kemudian melakukan pencarian terhadap nilai tertentu. Jika data ditemukan, akan ditampilkan bahwa data tersebut ada, dan jika tidak ditemukan akan ditampilkan bahwa data tidak ada.

Kesimpulannya, program ini menunjukkan cara melakukan pencarian data pada Single Linked List secara berurutan.

Proses pencarian dilakukan dari awal hingga akhir data, sehingga keberhasilan pencarian bergantung pada keberadaan nilai yang dicari dalam susunan data tersebut.

### d. HapusSLL\_2511532007

```

1 package pakej2_2511532007;
2
3 public class HapusSLL_2511532007 {
4     // fungsi untuk menghapus head
5     public static NodeSLL_2511532007 deleteHead(NodeSLL_2511532007 head) {
6         // jika list kosong
7         if (head == null) {
8             return null;
9         }
10        // hapus head ke node head.next
11        head = head.next;
12        // return head baru
13        return head;
14    }
15    // fungsi untuk menghapus node terakhir
16    public static NodeSLL_2511532007 removeLastNode(NodeSLL_2511532007 head) {
17        // jika list kosong - return null
18        if (head == null) {
19            return null;
20        }
21        // jika list satu node, hapus node dan return null
22        if (head.next == null) {
23            return null;
24        }
25        // hapus node terakhir
26        NodeSLL_2511532007 secondLast = head;
27        while (secondLast.next.next != null) {
28            secondLast = secondLast.next;
29        }
30        // hapus node terakhir
31        secondLast.next = null;
32        return head;
33    }
34 }

```

```

13 // fungsi menghapus node di posisi ke-1
14 public static NodeSLL_2511532007 deleteHead (NodeSLL_2511532007 head, int position) {
15     NodeSLL_2511532007 temp = head;
16     NodeSLL_2511532007 prev = null;
17     // jika list kosong
18     if (temp == null)
19         return head;
20     // kasus 1: head dihapus
21     if (position == 1) {
22         head = temp.next_2007;
23         return head;
24     }
25     // kasus 2: menghapus node di tengah
26     for (int i = 1; temp != null && i < position; i++) {
27         prev = temp;
28         temp = temp.next_2007;
29     }
30     // jika dihapus, fungsi node
31     if (temp != null) {
32         prev.next_2007 = temp.next_2007;
33     } else {
34         System.out.println("Data tidak ada");
35         return head;
36     }
37     // fungsi menampilkan list
38     public static void printList(NodeSLL_2511532007 head) {
39         NodeSLL_2511532007 cur = head;
40         while (cur.next_2007 != null) {
41             System.out.print(cur.data_2007 + " ");
42             if (cur.next_2007 == null) {
43                 if (cur.next_2007 == null) {
44                     System.out.print(cur.data_2007);
45                     System.out.println();
46                 }
47             }
48         }
49     }
50 }
51
52 // fungsi main
53 public static void main (String[] args) {
54     // head SLL 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100
55     NodeSLL_2511532007 head = new NodeSLL_2511532007(1);
56     head.next_2007 = new NodeSLL_2511532007(2);
57     head.next_2007.next_2007 = new NodeSLL_2511532007(3);
58     head.next_2007.next_2007.next_2007 = new NodeSLL_2511532007(4);
59     head.next_2007.next_2007.next_2007.next_2007 = new NodeSLL_2511532007(5);
60     head.next_2007.next_2007.next_2007.next_2007.next_2007 = new NodeSLL_2511532007(6);
61     // cetak list awal
62     System.out.println("list awal: ");
63     printList(head);
64     // hapus head
65     head = deleteHead(head);
66     System.out.println("list setelah head dihapus: ");
67     printList(head);
68     // hapus node di position 2
69     int position = 2;
70     // hapus node ke-2
71     head = deleteHead(head, position);
72     // print list setelah node ke-2 dihapus
73     System.out.println("list setelah posisi 2 dihapus: ");
74     printList(head);
75 }
76 }
77 }
78 }
79 }
80 }
81 }
82 }
83 }
84 }
85 }
86 }
87 }
88 }
89 }
90 }
91 }
92 }
93 }
94 }
95 }
96 }
97 }
98 }
99 }
100 }

```

Langkah-langkah susunan kode program beserta fungsinya:

1. Deklarasi awal program (package dan class)  
Program diawali dengan package pekan5\_2511532007; untuk mengelompokkan file Java. Selanjutnya dibuat public class HapusSLL\_2511532007 sebagai kelas utama yang berisi implementasi penghapusan data pada Single Linked List.
2. Method deleteHead() (menghapus data pertama)  
Method ini digunakan untuk menghapus data pada bagian awal linked list. Jika data masih kosong, maka tidak ada yang dihapus. Jika terdapat data, maka data pertama akan dihilangkan dan posisi awal akan berpindah ke data berikutnya.
3. Method removeLastNode() (menghapus data terakhir)  
Method ini digunakan untuk menghapus data di bagian akhir linked list. Jika tidak ada data, maka tidak terjadi perubahan. Jika hanya terdapat satu data, maka setelah dihapus linked list menjadi kosong. Jika terdapat lebih dari satu data, program akan mencari data terakhir kedua, kemudian menghapus data terakhir.

4. Method `deleteNode()` (menghapus data pada posisi tertentu)  
Method ini digunakan untuk menghapus data berdasarkan posisi tertentu. Jika posisi pertama dipilih, maka data awal akan dihapus. Jika posisi lainnya, program akan menelusuri hingga posisi yang dimaksud, kemudian menghapus data tersebut. Jika posisi tidak ditemukan, maka akan ditampilkan pesan bahwa data tidak ada.
5. Method `printList()` (menampilkan isi linked list)  
Method ini digunakan untuk menampilkan seluruh data dalam linked list secara berurutan dari awal hingga akhir.
6. Method `main()` (program utama)  
Method `main` digunakan untuk menjalankan program. Pada bagian ini dibuat data awal yaitu 1, 2, 3, 4, 5, dan 6. Data tersebut kemudian ditampilkan. Setelah itu dilakukan proses penghapusan data pertama, lalu ditampilkan hasilnya. Selanjutnya dilakukan penghapusan data pada posisi tertentu (posisi ke-2), dan hasilnya kembali ditampilkan.
7. Alur kerja program  
Program dimulai dengan menampilkan data awal, kemudian melakukan beberapa proses penghapusan, yaitu menghapus data di awal dan pada posisi tertentu. Setelah setiap proses, hasil susunan data akan diperbarui dan ditampilkan kembali.

Kesimpulannya, program ini menunjukkan proses penghapusan data pada Single Linked List, baik di bagian awal, akhir, maupun pada posisi tertentu. Dengan cara ini, data dapat dikelola secara dinamis sesuai kebutuhan tanpa bergantung pada ukuran tetap.

## **BAB III**

### **KESIMPULAN DAN SARAN**

#### **3.1 Kesimpulan**

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Single Linked List merupakan struktur data yang mampu menyimpan dan mengelola data secara dinamis. Melalui implementasi program, terlihat bahwa operasi dasar seperti penambahan, pencarian, dan penghapusan data dapat dilakukan dengan mengikuti urutan data dari awal hingga akhir. Struktur ini memberikan fleksibilitas dalam pengolahan data karena tidak bergantung pada ukuran tetap. Namun, proses akses data membutuhkan penelusuran secara berurutan sehingga dapat mempengaruhi efisiensi ketika jumlah data semakin banyak.

#### **3.2 Saran**

Dalam pelaksanaan praktikum, sebaiknya dilakukan pemahaman konsep terlebih dahulu sebelum mengimplementasikan kode program agar tidak hanya sekadar mengikuti langkah-langkah. Selain itu, perlu dilakukan pengujian program dengan berbagai kondisi, seperti data kosong atau posisi yang tidak valid, untuk memastikan program berjalan dengan baik.

## DAFTAR PUSTAKA

- [1] Wikipedia, “Linked list,” 2026. [Online]. Available: [https://en.wikipedia.org/wiki/Linked\\_list](https://en.wikipedia.org/wiki/Linked_list) [Diakses 4 Mei 2026]
- [2] Trivusi, “Struktur Data Linked List: Pengertian dan Karakteristik,” 2022. [Online]. Available: <https://www.trivusi.web.id/2022/07/struktur-data-linked-list.html> [Diakses 5 Mei 2026]
- [3] Carnegie Mellon University, “Linked Lists,” 2024. [Online]. Available: <https://web2.qatar.cmu.edu/cs/15121/notes/linkedlist/> [Diakses 5 Mei 2026]
- [4] DigitalOcean, “Java LinkedList,” 2022. [Online]. Available: <https://www.digitalocean.com/community/tutorials/java-linkedlist-linkedlist-java> [Diakses 6 Mei 2026]

## TUGAS PRAKTIKUM PEKAN 5

### 1. Kelas Pasien

```
package pekan5_2511532007;

import java.util.*;

public class Pasien_2511532007 {
    private String namaPasien_2007;
    private String keluhan_2007;
    private int nomorAntrian_2007;
    private Pasien_2511532007 next_2007;

    // Constructor
    public Pasien_2511532007(String namaPasien_2007, String
keluhan_2007, int nomorAntrian_2007) {
        this.namaPasien_2007 = namaPasien_2007;
        this.keluhan_2007 = keluhan_2007;
        this.nomorAntrian_2007 = nomorAntrian_2007;
        this.next_2007 = null;
    }

    // Getter
    public String getNamaPasien_2007() {
        return namaPasien_2007;
    }

    public String getKeluhan_2007() {
        return keluhan_2007;
    }

    public int getNomorAntrian_2007() {
        return nomorAntrian_2007;
    }

    public Pasien_2511532007 getNext_2007() {
        return next_2007;
    }
}
```



```

// Setter
public void setNamaPasien_2007(String namaPasien_2007) {
    this.namaPasien_2007 = namaPasien_2007;
}

public void setKeluhan_2007(String keluhan_2007) {
    this.keluhan_2007 = keluhan_2007;
}

public void setNomorAntrian_2007(int nomorAntrian_2007) {
    this.nomorAntrian_2007 = nomorAntrian_2007;
}

public void setNext_2007(Pasien_2511532007 next_2007) {
    this.next_2007 = next_2007;
}
}

```

## 2. Kelas RumahSakit

```

package pekan5_2511532007;

import java.util.Scanner;

public class RumahSakit_2511532007 {

    private Pasien_2511532007 head_2007;

    private int counter_2007 = 0;

    public void daftarkanPasien_2007(String nama_2007, String
keluhan_2007) {

        counter_2007++;

        Pasien_2511532007 pasienBaru_2007 =

            new Pasien_2511532007(nama_2007, keluhan_2007,
counter_2007);

        if (head_2007 == null) {

```

```

        head_2007 = pasienBaru_2007;
    } else {
        Pasien_2511532007 current_2007 = head_2007;
        while (current_2007.getNext_2007() != null) {
            current_2007 = current_2007.getNext_2007();
        }
        current_2007.setNext_2007(pasienBaru_2007);
    }

    System.out.println("Pasien berhasil didaftarkan! Nomor Antrian: " +
counter_2007);
}

public void panggilPasien_2007() {
    if (head_2007 == null) {
        System.out.println("Antrian kosong.");
        return;
    }

    System.out.println("Pasien dipanggil.");
    System.out.println("Nama: " + head_2007.getNamaPasien_2007());
    System.out.println("Keluhan: " + head_2007.getKeluhan_2007());
    System.out.println("Nomor Antrian: " +
head_2007.getNomorAntrian_2007());

    head_2007 = head_2007.getNext_2007();
}

```

```

public void tampilkanAntrian_2007() {
    if (head_2007 == null) {
        System.out.println("Antrian kosong.");
        return;
    }

    Pasien_2511532007 current_2007 = head_2007;

    System.out.println("\n=== Daftar Antrian Pasien ===");
    while (current_2007 != null) {
        System.out.println("No Antrian : " +
            current_2007.getNomorAntrian_2007());
        System.out.println("Nama      : " +
            current_2007.getNamaPasien_2007());
        System.out.println("Keluhan   : " +
            current_2007.getKeluhan_2007());
        System.out.println("-----");

        current_2007 = current_2007.getNext_2007();
    }
}

public void cariPasien_2007(String namaCari_2007) {
    if (head_2007 == null) {
        System.out.println("Antrian kosong.");
        return;
    }
}

```

```

    Pasien_2511532007 current_2007 = head_2007;

    boolean ditemukan_2007 = false;

    while (current_2007 != null) {
        if
        (current_2007.getNamaPasien_2007().equalsIgnoreCase(namaCari_2007))
        {
            System.out.println("Pasien ditemukan!");

            System.out.println("Nama: " +
            current_2007.getNamaPasien_2007());

            System.out.println("Keluhan: " +
            current_2007.getKeluhan_2007());

            System.out.println("Nomor Antrian: " +
            current_2007.getNomorAntrian_2007());

            ditemukan_2007 = true;

            break;
        }

        current_2007 = current_2007.getNext_2007();
    }

    if (!ditemukan_2007) {
        System.out.println("Pasien tidak ditemukan.");
    }
}

public void cekStatusAntrian_2007() {
    if (head_2007 == null) {

```

```

        System.out.println("Antrian kosong.");
        return;
    }

    int jumlah_2007 = 0;
    Pasien_2511532007 current_2007 = head_2007;

    while (current_2007 != null) {
        jumlah_2007++;
        current_2007 = current_2007.getNext_2007();
    }

    System.out.println("Total pasien dalam antrian: " + jumlah_2007);
    System.out.println("Pasien terdepan: " +
        head_2007.getNamaPasien_2007());
    }

    public static void main(String[] args) {
        Scanner input_2007 = new Scanner(System.in);
        RumahSakit_2511532007 rs_2007 = new RumahSakit_2511532007();

        int pilihan_2007;

        do {
            System.out.println("\n==== Antrian Rumah Sakit NIM: 2511532007
            ===");
            System.out.println("1. Daftarkan Pasien");
            System.out.println("2. Panggil Pasien");

```

```
System.out.println("3. Tampilkan Antrian");
System.out.println("4. Cari Pasien");
System.out.println("5. Cek Status Antrian");
System.out.println("6. Keluar");
System.out.print("Pilihan: ");
pilihan_2007 = input_2007.nextInt();
input_2007.nextLine();

switch (pilihan_2007) {
    case 1:
        System.out.print("Masukkan Nama Pasien: ");
        String nama_2007 = input_2007.nextLine();

        System.out.print("Masukkan Keluhan: ");
        String keluhan_2007 = input_2007.nextLine();

        rs_2007.daftarkanPasien_2007(nama_2007, keluhan_2007);
        break;

    case 2:
        rs_2007.panggilPasien_2007();
        break;

    case 3:
        rs_2007.tampilkanAntrian_2007();
        break;

    case 4:
```

```
System.out.print("Masukkan nama pasien yang dicari: ");
```

```
String cari_2007 = input_2007.nextLine();
```

```
rs_2007.cariPasien_2007(cari_2007);
```

```
break;
```

case 5:

```
rs_2007.cekStatusAntrian_2007();
```

```
break;
```

case 6:

```
System.out.println("Program selesai.");
```

```
break;
```

default:

```
System.out.println("Pilihan tidak valid.");
```

```
}
```

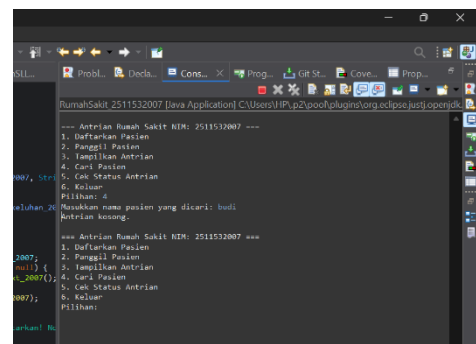
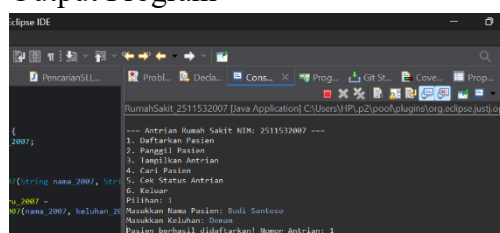
```
} while (pilihan_2007 != 6);
```

```
input_2007.close();
```

```
}
```

```
}
```

### 3. Output Program



Kesimpulannya, Program Simulasi Antrian Rumah Sakit menggunakan Single Linked List berhasil mengimplementasikan konsep struktur data linear dinamis dengan sistem FIFO (First In First Out), di mana pasien yang pertama mendaftar akan dipanggil lebih dahulu. Setiap pasien disimpan dalam bentuk node yang berisi data nama pasien, keluhan, nomor antrian, serta pointer menuju node berikutnya.

Program ini memiliki beberapa fitur utama yaitu menambahkan pasien ke akhir antrian (insert at tail), memanggil pasien terdepan (delete head), menampilkan seluruh daftar antrian, mencari data pasien berdasarkan nama secara case-insensitive, serta mengecek status antrian. Selain itu, program juga mampu menangani kondisi ketika antrian kosong agar tidak terjadi error.

Dari program ini dapat disimpulkan bahwa penggunaan Single Linked List sangat efektif untuk mengelola data antrian secara dinamis karena ukuran data dapat bertambah atau berkurang sesuai kebutuhan tanpa harus menentukan kapasitas array di awal.